TOPIC 1

Gradient Descent Basics: Explained

The original class slides, with a plain explanation under each one

> **NOTE** **How to read this document**
>
> - Every original slide is kept exactly as it was, with its slide number under it
> - Under each slide (or small set of slides) you get a plain explanation: **what you are looking at** and **what it means**
> - The slides come from a recorded class, so a few look like screenshots of a video: that is normal

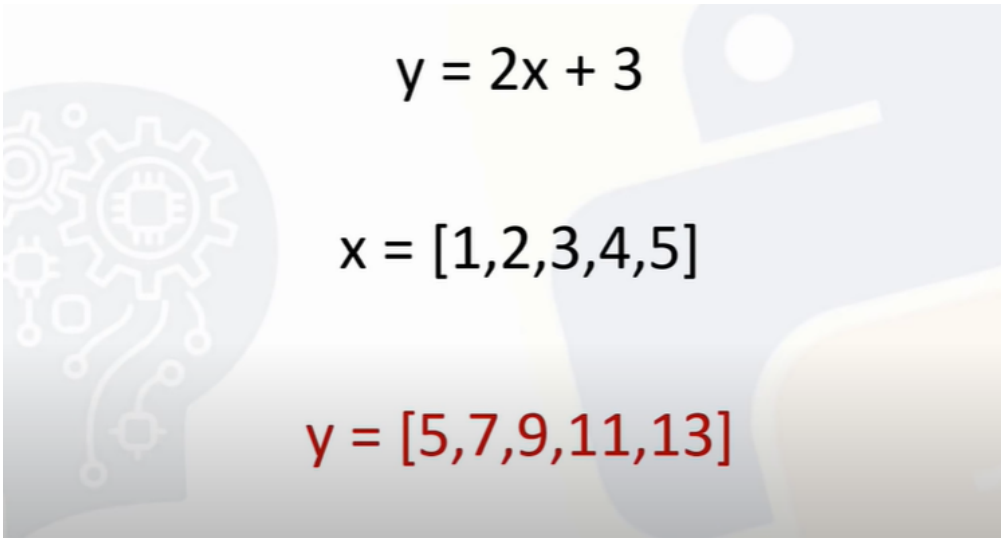
These 33 slides introduce **gradient descent** with one running example: predicting the **price of a house from its area**. The big question the slides answer is: *how does a computer find the best straight line through a set of points?*

The idea in one sentence: start with a guess for the line, measure how wrong it is, and keep nudging it in the direction that makes it less wrong, until it cannot improve any more.

Words you will meet

Word	Plain meaning
Model / formula	A rule that turns inputs into a prediction, such as y = 2x + 3
Slope (m)	How steeply the line rises: how much y grows when x grows by 1
Intercept (b)	Where the line crosses the y-axis (the value of y when x = 0)
Error (Δ)	The gap between a real value and the line's prediction
MSE (cost)	Mean Squared Error: the average of the squared errors. Smaller means a better line
Gradient descent	Repeatedly stepping downhill on the cost until the lowest point is found
Learning rate	How big each step is
Global minimum	The lowest point of the cost: the best possible line

Slide 1: From a formula to answers

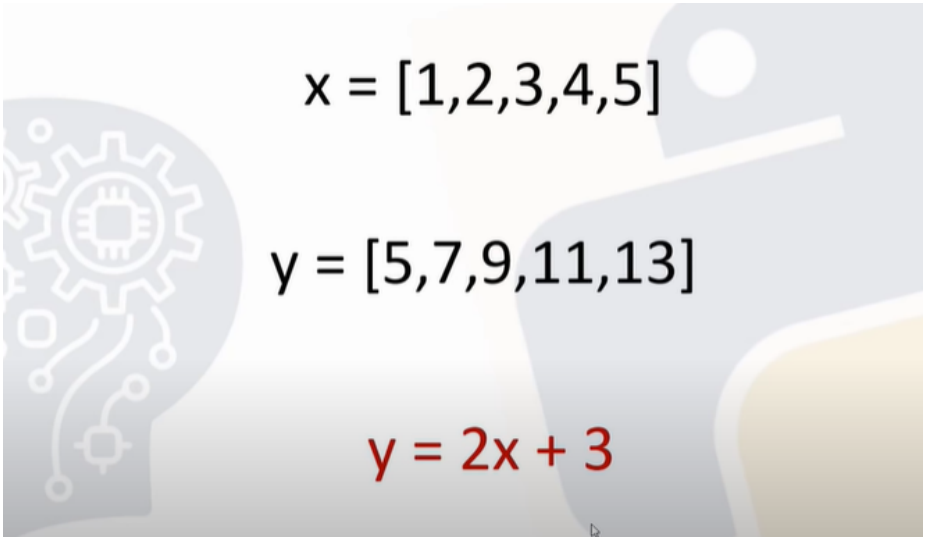


Slide 1

This slide shows an ordinary formula, **y = 2x + 3**, and a list of inputs, x = [1, 2, 3, 4, 5]. If you put each x into the formula you get the outputs y = [5, 7, 9, 11, 13] (for example, x = 1 gives 2 × 1 + 3 = 5).

- Formula + inputs gives outputs. This is the normal direction of maths.

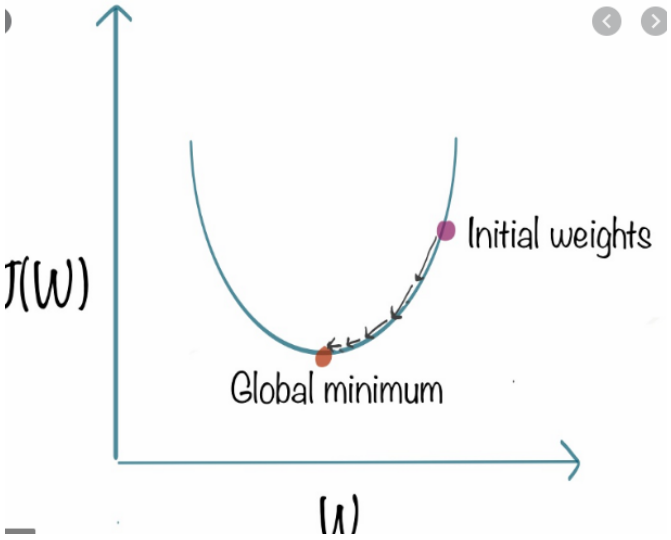
Slide 2: Now the other way round: find the formula



Slide 2

Here the lists come first: x = [1, 2, 3, 4, 5] and y = [5, 7, 9, 11, 13], and the formula y = 2x + 3 is the *unknown* answer in red. **This is what machine learning does:** it is given inputs and outputs and has to discover the rule that connects them. Gradient descent is one way to discover it.

Slide 3: The picture to keep in mind: a bowl

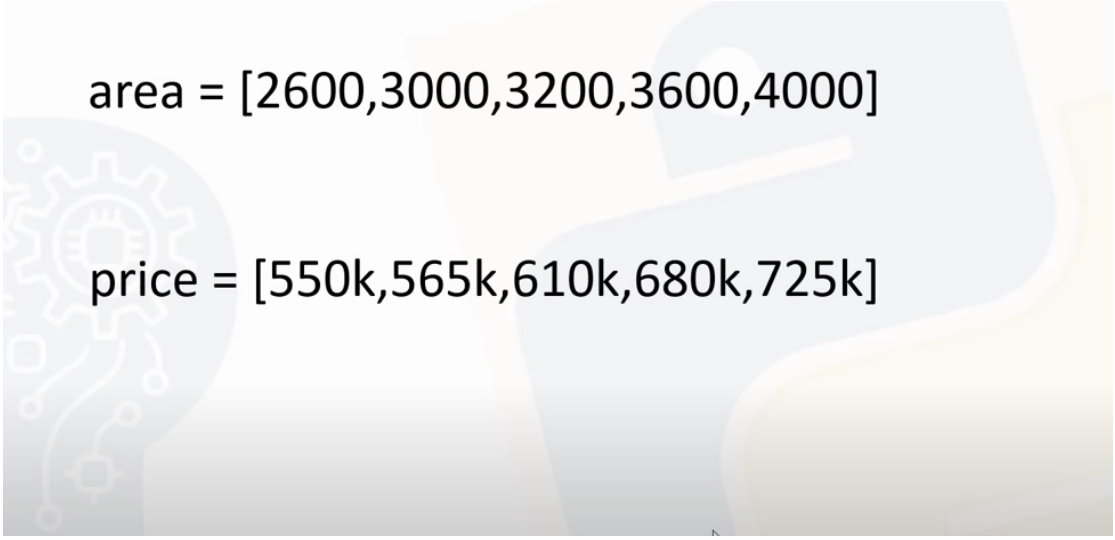


Slide 3

The curve is a bowl. The horizontal axis is the value of a weight (a number the model can change, such as the slope). The vertical axis, J(W), is the **cost**: how wrong the model is. The purple dot is the **initial weights**, our first random guess. The arrows show the dot sliding down the bowl to the **global minimum**, the lowest point, where the model is least wrong.

- High on the bowl = bad model. Bottom of the bowl = best model.
- Gradient descent is simply 'keep walking downhill'.

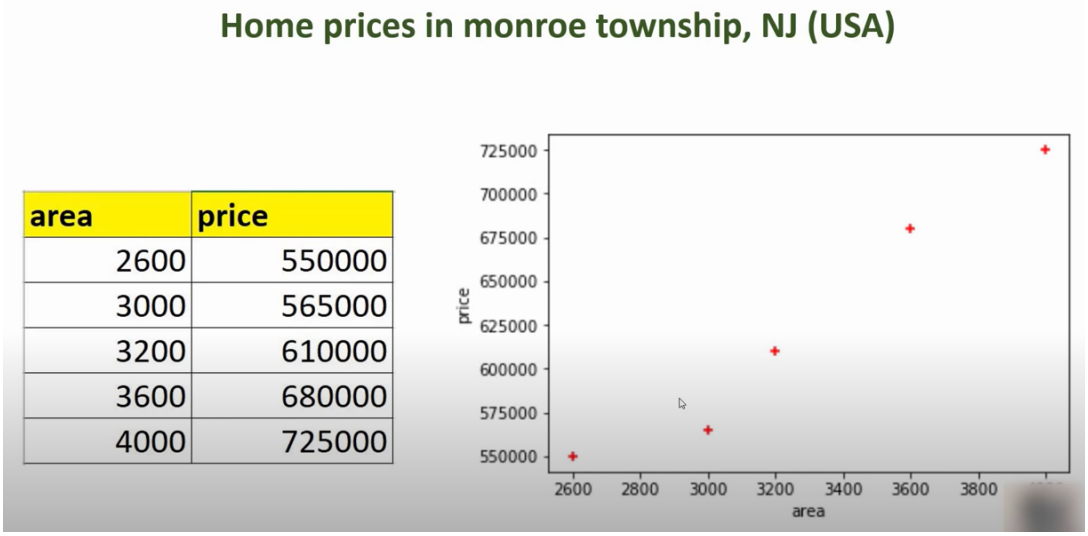
Slide 4: Our data: house area and price



Slide 4

Two lists: the **area** of five houses in square feet (2600, 3000, 3200, 3600, 4000) and their **price** (550k, 565k, 610k, 680k, 725k dollars). We want a rule that predicts the price from the area.

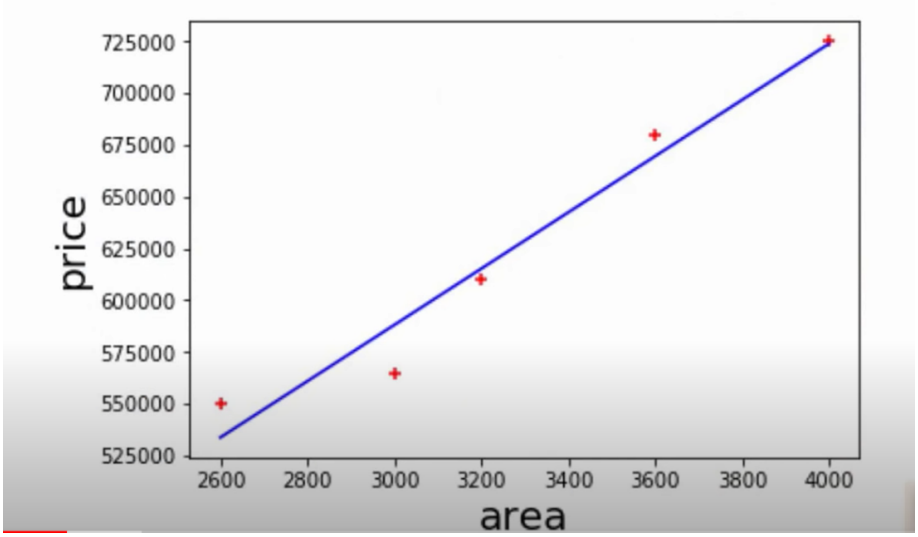
Slide 5: The data as a table and a scatter plot



Slide 5

The same five houses, now as a table (area, price) and as a scatter plot with one red dot per house. These are home prices in Monroe Township, New Jersey. The dots go up to the right: bigger houses cost more. A straight line should describe that trend well.

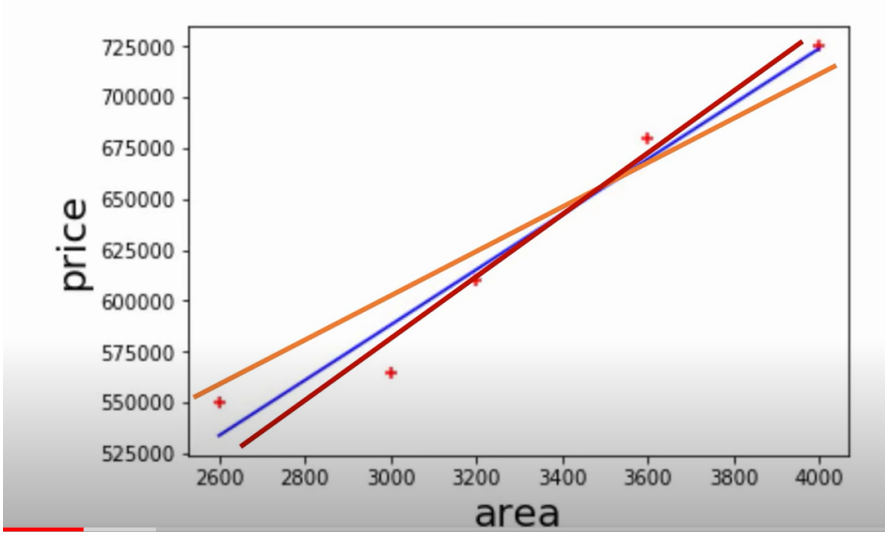
Slide 6: A line through the points



Slide 6

A blue straight line drawn through the red dots. It does not touch every dot but it follows the trend. A line like this is written **price = m × area + b**, where m is the slope and b the intercept. The task is to find the best m and b.

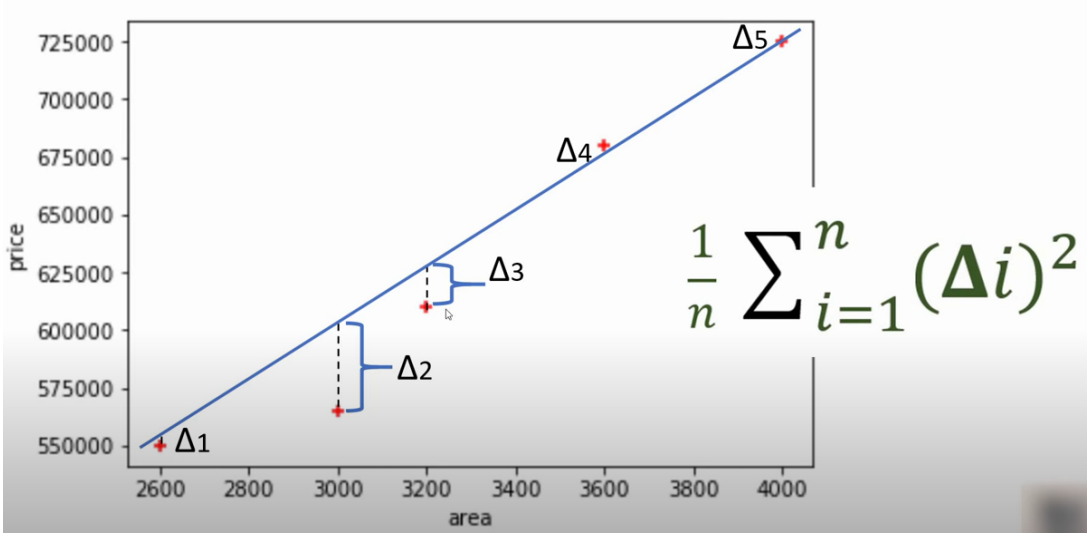
Slide 7: Many possible lines: which is best?



Slide 7

Three different lines (orange, blue and dark red) all go roughly through the dots. Each has a different slope and intercept. We need a number that says **how good a line is**, so we can pick the best one automatically. That number is the cost.

Slide 8: Measuring the error of a line

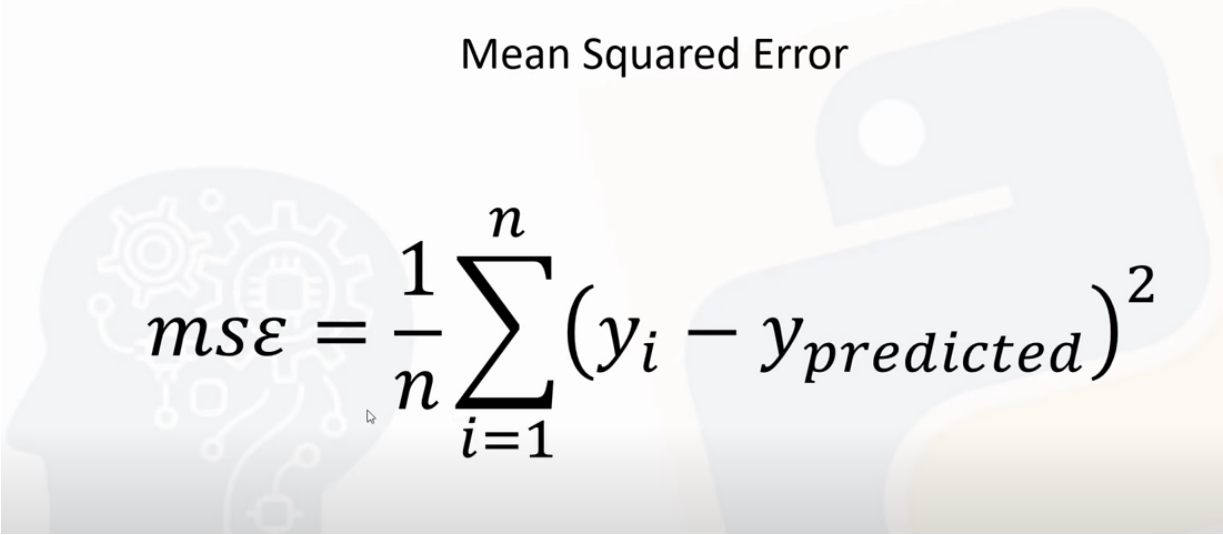


Slide 8

The small marks Δ1 to Δ5 are the vertical gaps between each real price (red dot) and the line's prediction. Each gap is an **error**. Some dots are above the line and some below, so the errors are **squared** (which makes them all positive and punishes big misses more) and then averaged, as shown by the formula $1/n \times \Sigma\ (\Delta i)^2$.

$$\frac{1}{n}\sum_{i=1}^{n}(\Delta_i)^2$$

Slide 9: The Mean Squared Error formula

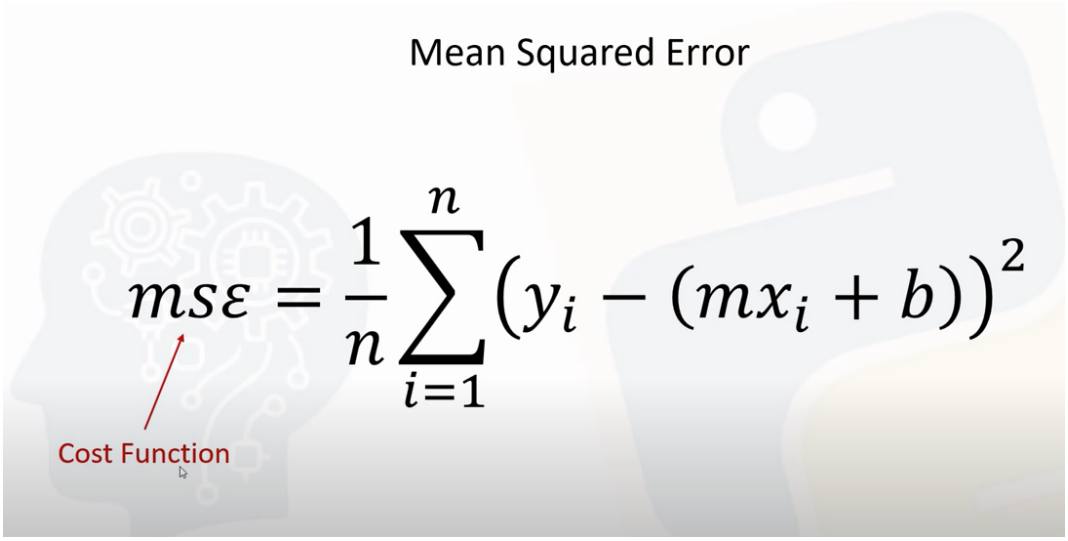


Slide 9

The same idea written properly. For each house i, take the real price y_i, subtract the predicted price, square the difference, add them all up (Σ), and divide by the number of houses n.

- **y_i**: the real value
- **y_predicted**: what the line says
- **n**: how many points there are

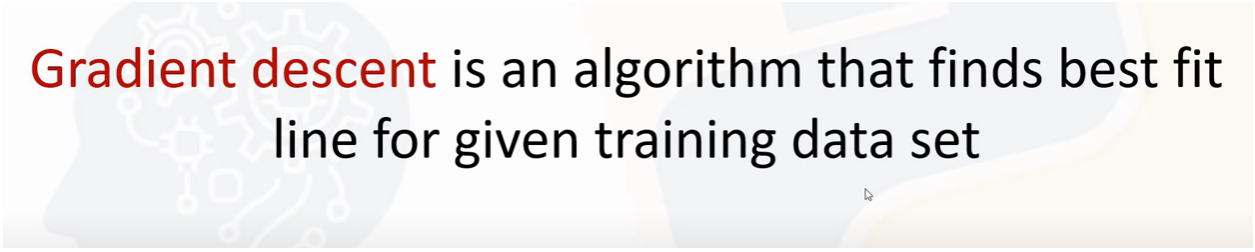
Slide 10: MSE written with the line inside: the cost function



Slide 10

The prediction is now replaced with the line itself: m × x_i + b. So the error depends on the two numbers we can change, m and b. This formula is called the **cost function**. Pick a different m and b and you get a different cost. Our goal: find the m and b that make the cost as small as possible.

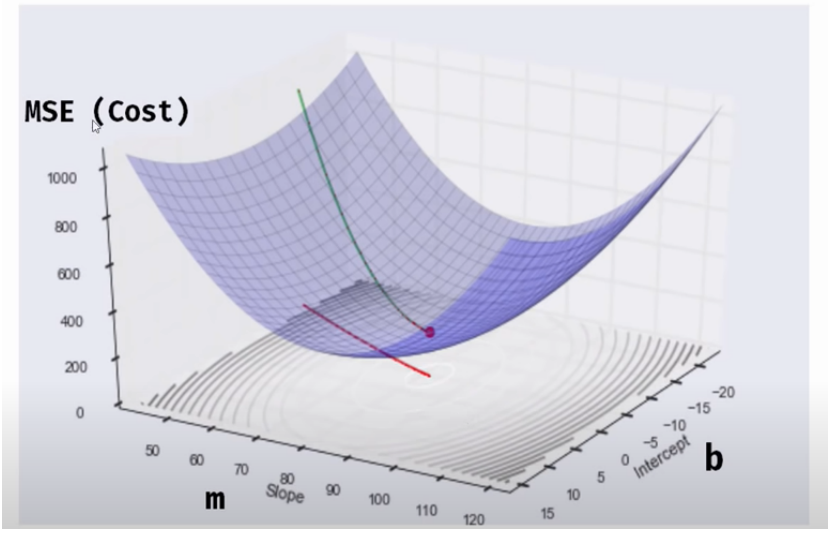
Slide 11: The definition



Slide 11

Gradient descent is an algorithm that finds the best-fit line for a given training data set. It does this by repeatedly changing m and b to reduce the cost.

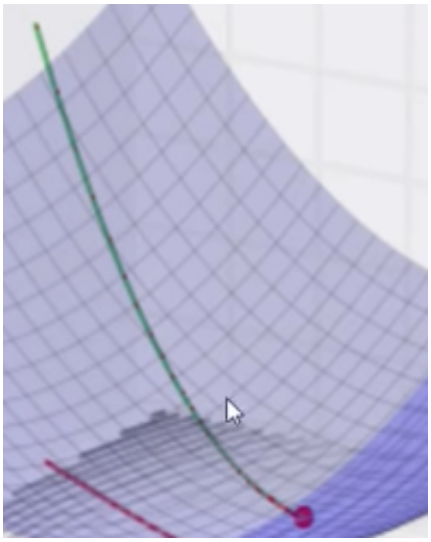
Slide 12: The cost as a 3D bowl



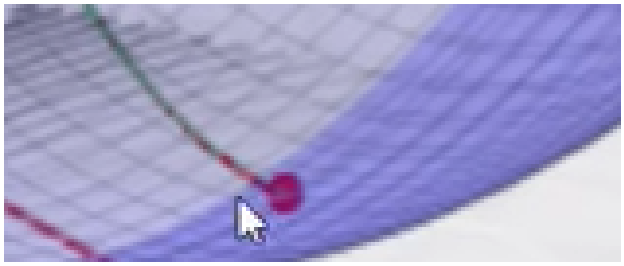
Slide 12

Now there are two things to adjust, so the cost needs a 3D picture. The two horizontal axes are the **slope m** and the **intercept b**. The height is the **MSE (cost)**. Every point on the surface is one possible line, and its height tells you how bad that line is. The surface is a smooth bowl, and the best line is at the very bottom.

Slides 13 to 14: Zoom into the bowl: the path downhill



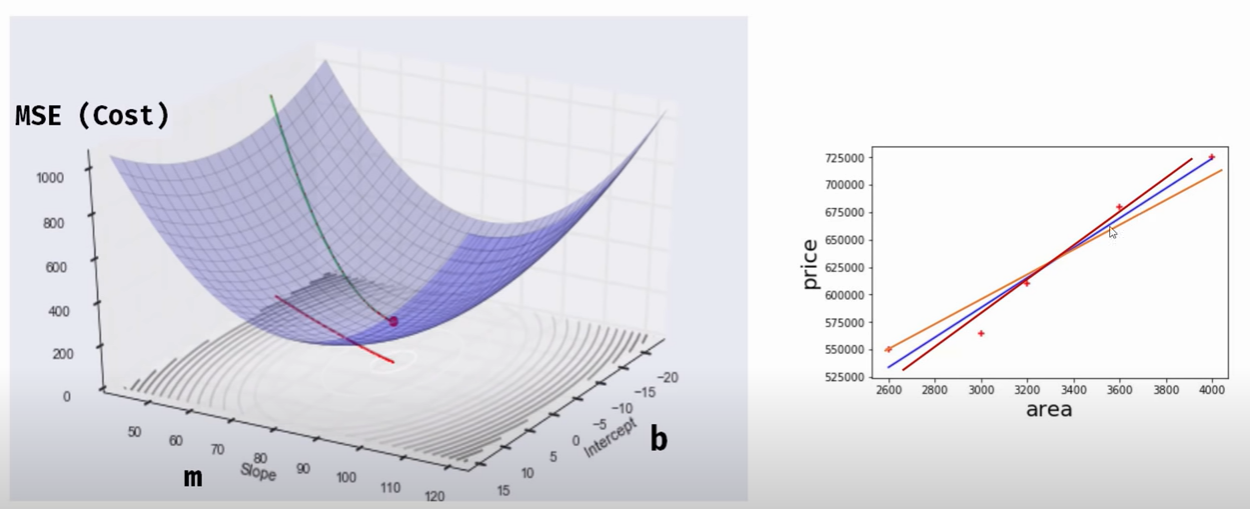
Slide 13



Slide 14

Two zoomed views of the bowl. The green curve is the path taken by the purple dot (our current guess) as it rolls towards the bottom. The red line on the floor of the picture shows where that path sits in the (m, b) plane.

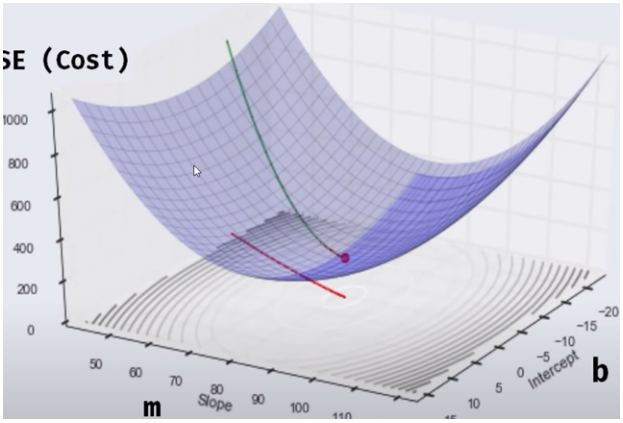
Slide 15: Bowl and lines side by side



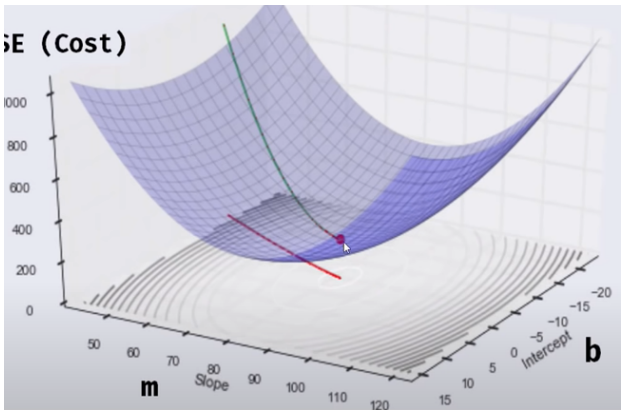
Slide 15

Left: the cost bowl. Right: the house data with the candidate lines. This slide connects the two ideas: **a point in the bowl is a line in the data plot**. As the dot moves down the bowl, the line fits the data better.

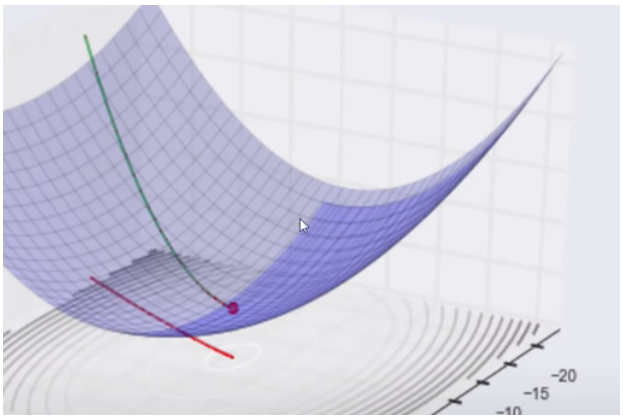
Slides 16 to 18: The dot rolling down the bowl



Slide 16



Slide 17

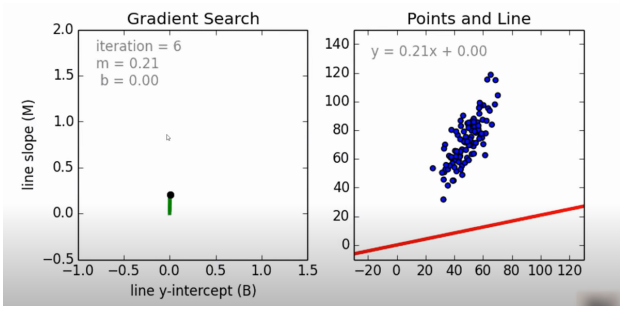


Slide 18

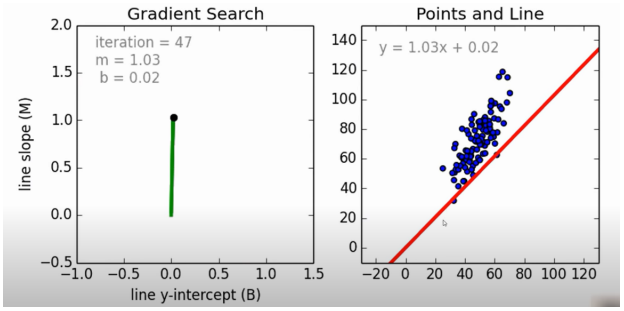
Three frames of the same animation. In each, the purple dot sits a little lower on the surface. Lower means a smaller cost, which means a better line. The bottom of the bowl is the best line,

the **global minimum**.

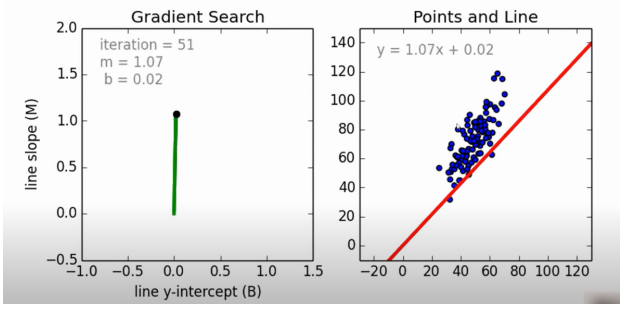
Slides 19 to 22: Watching the search: early iterations



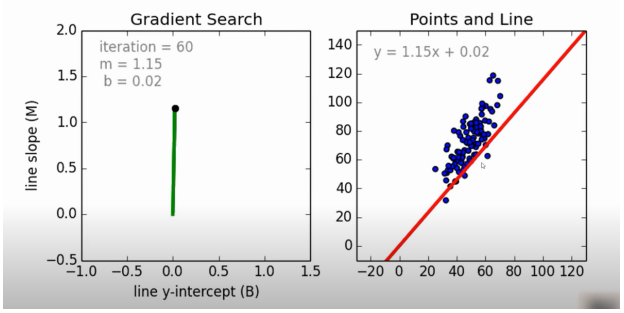
Slide 19



Slide 20



Slide 21

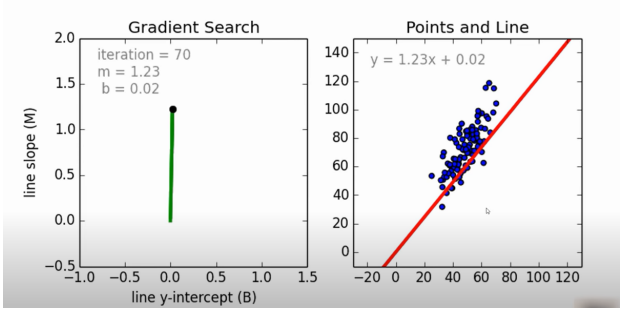


Slide 22

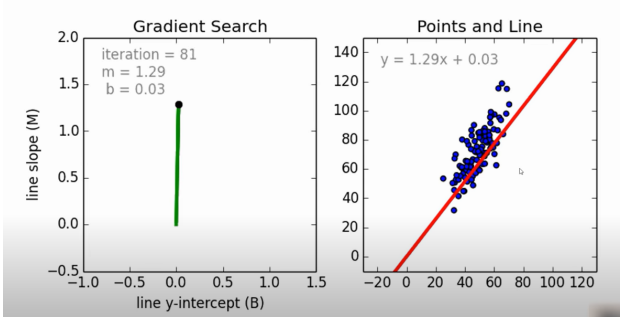
Each slide has two plots. **Left (Gradient Search):** the guess for the slope (vertical axis) and the intercept (horizontal axis); the black dot is the current guess and the green line is where it has travelled. **Right (Points and Line):** the data points (blue) and the current line (red). One *iteration* is one step of gradient descent.

- Iteration 6: slope m = 0.21, so the line is almost flat and far from the points
- Iteration 47: m = 1.03, the line is already close to the cloud of points
- Iteration 51: m = 1.07
- Iteration 60: m = 1.15, the line is climbing with the data

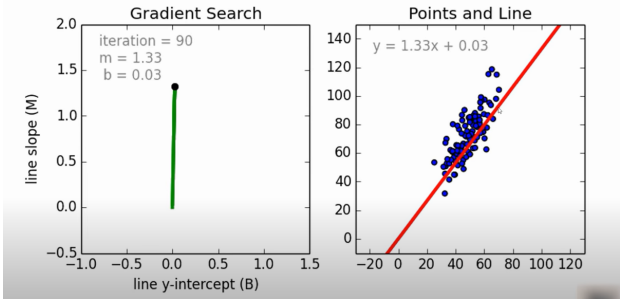
Slides 23 to 26: The middle of the search



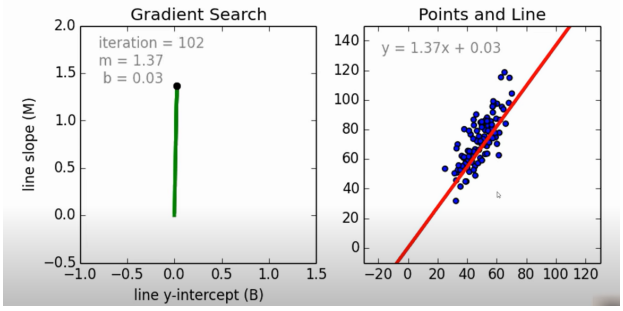
Slide 23



Slide 24



Slide 25

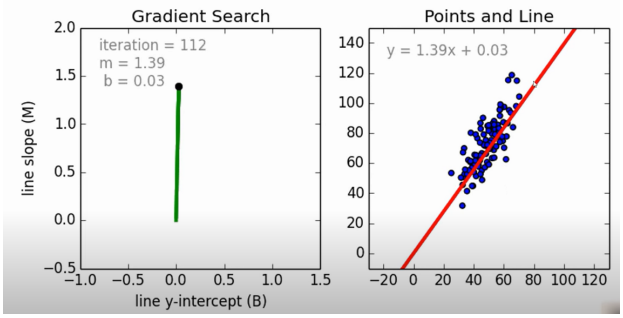


Slide 26

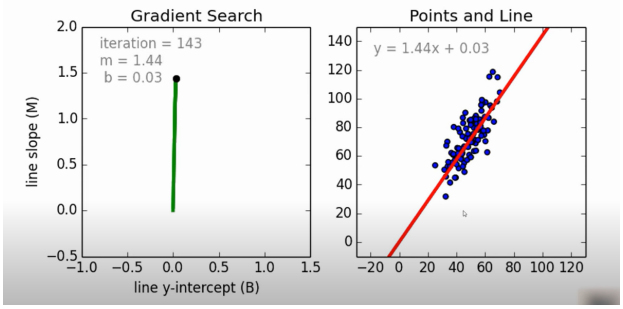
The same pair of plots at later iterations. The slope keeps creeping upward and the red line rotates until it runs through the middle of the cloud.

- Iteration 70: m = 1.23
- Iteration 81: m = 1.29
- Iteration 90: m = 1.33
- Iteration 102: m = 1.37

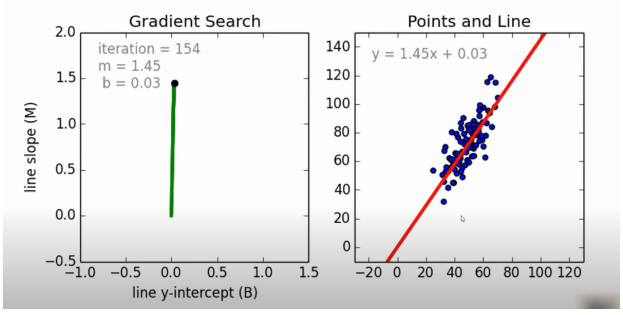
Slides 27 to 29: The end of the search: tiny changes



Slide 27



Slide 28

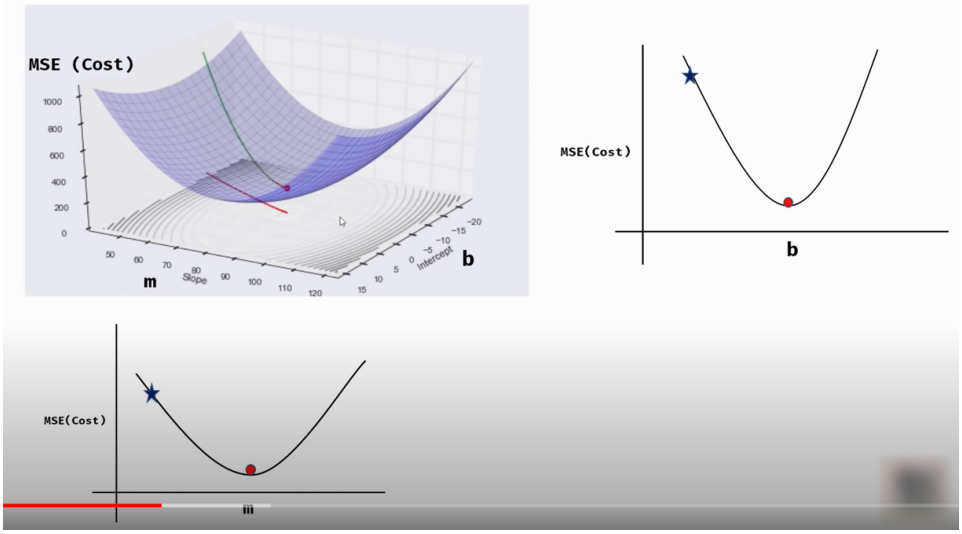


Slide 29

Near the end the line barely moves. This is how you know gradient descent is finishing: the steps become very small because the cost surface is almost flat near the bottom.

- Iteration 112: m = 1.39
- Iteration 143: m = 1.44
- Iteration 154: m = 1.45 and b = 0.03

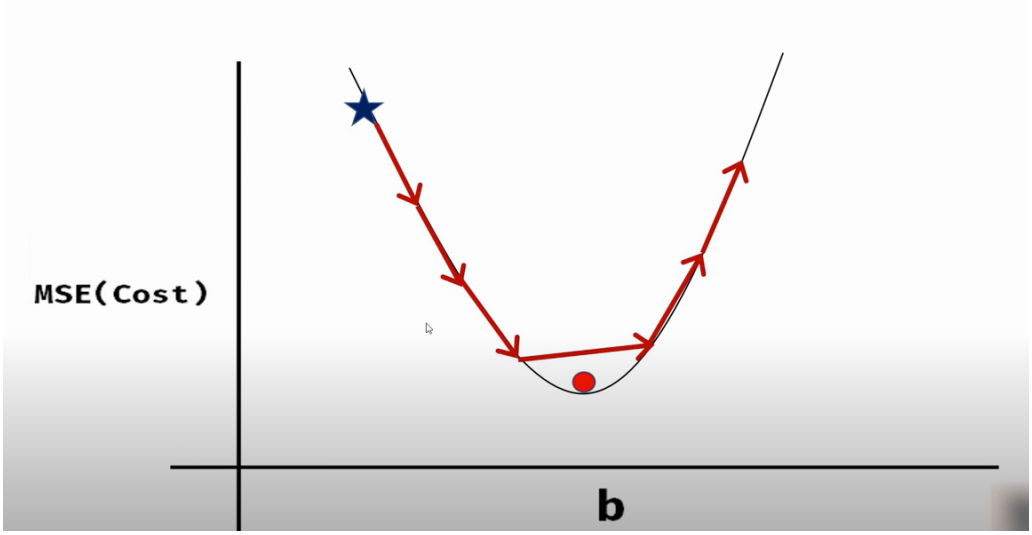
Slide 30: One parameter at a time



Slide 30

Top left: the 3D bowl. The two smaller curves are slices through it: the **MSE against b** (right) and the **MSE against m** (bottom). In each slice the blue star is the starting guess and the red dot is the best value. It is easier to understand how the descent works on one curve at a time.

Slide 31: Steps that are too big

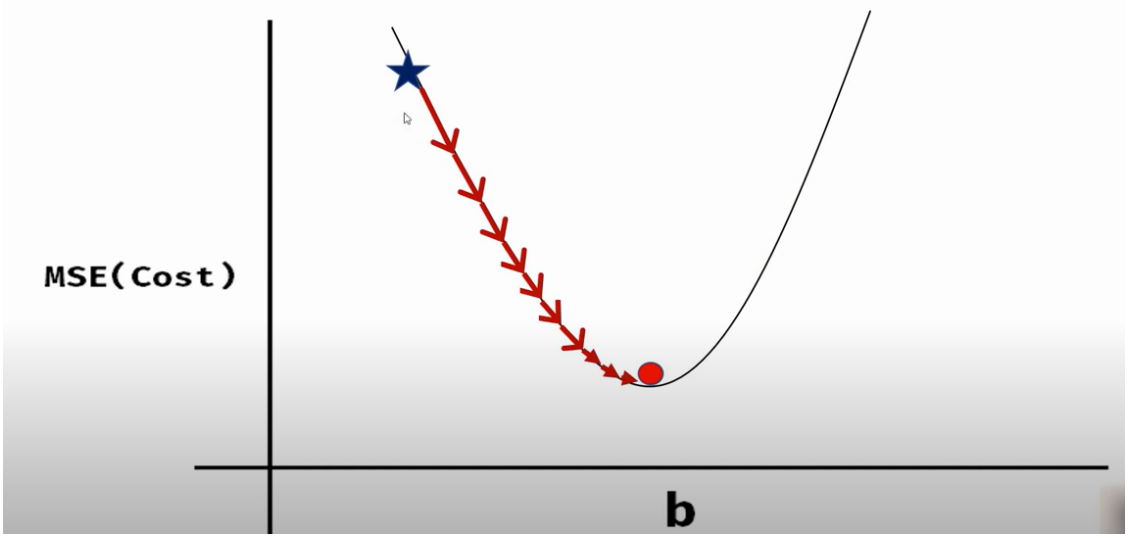


Slide 31

The red arrows are the steps of gradient descent on the cost curve. Here every step is **too large**, so the dot jumps right over the bottom and lands on the other side, then back again. If the steps are too big the search overshoots the minimum and may never settle.

- Cause: a **learning rate** that is too high

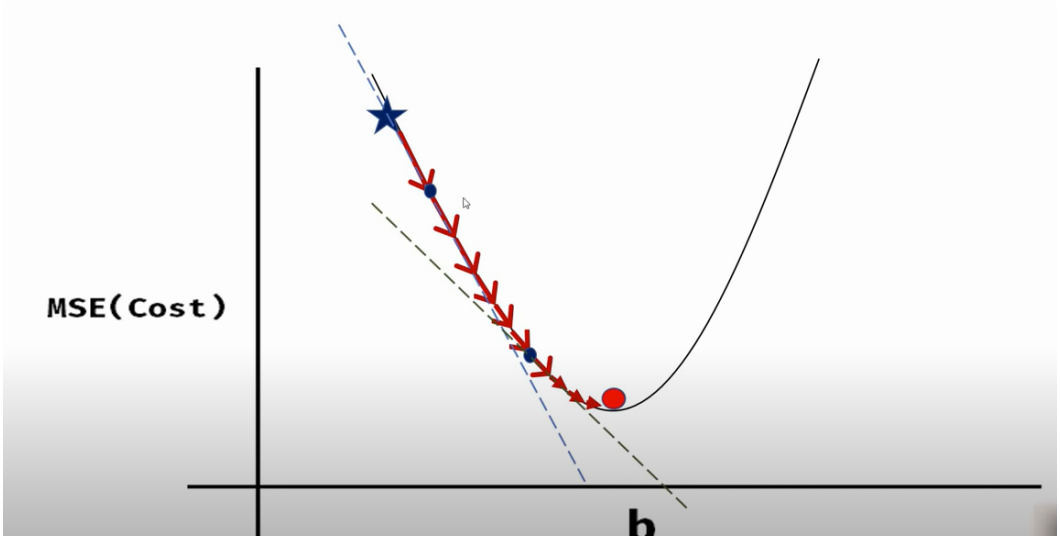
Slide 32: Steps that are tiny



Slide 32

Here the steps are very small, so the dot does walk down to the red dot but takes a very long time. Too small a learning rate means slow progress and lots of wasted computation.

Slide 33: Steps that shrink on their own



Slide 33

The dashed lines show the **slope** of the curve at two places. Steep ground means big steps; flat ground near the bottom means small steps. So even with a fixed learning rate the steps naturally get smaller as the dot approaches the minimum, which lets it settle on the answer.

- Slope steep: take a big step
- Slope flat: take a small step
- Slope zero: you are at the bottom, stop